

PROGRAMMAZIONE II

VARIABILI JAVA E CALL STACK

Michele Pasqua, PhD



UNIVERSITÀ
di **VERONA**
Dipartimento
di **INFORMATICA**

A.A. 2025/2026

TIPI DI VARIABILE

x_1

MEMORIA DI PROGRAMMA

Memoria statica: usata per gli elementi condivisi da tutte le istanze della classe

- ▶ Come le definizioni delle classi e i campi `static`

Memoria heap: usata per gli elementi riservati alle istanze della classe

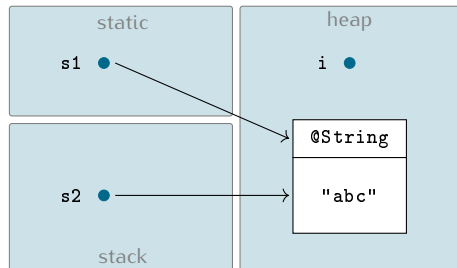
- ▶ Come gli oggetti creati con `new`
- ▶ Anche i campi di istanza sono memorizzati nell'heap, alla creazione dell'oggetto

Memoria stack: usata per gli elementi creati a runtime

- ▶ Come le variabili locali e i parametri dei metodi

TIPI DI MEMORIA DI PROGRAMMA (CONT.)

```
public class Clazz {  
    int i;  
    static String s1;  
    void aMethod() {  
        String s2;  
        s2 = new String("abc");  
        s1 = s2;  
    }  
}
```



VARIABILI DI ISTANZA

Le variabili di istanza sono memorizzate nell'heap

- ▶ Chiamate anche campi della classe

Ne esiste una copia diversa in ogni istanza della classe

- ▶ Create e inizializzate ogni volta che viene creato un oggetto
- ▶ Sempre (implicitamente) inizializzate

// nessun errore di compilazione

Le variabili di istanza **non sono condivise** tra oggetti della stessa classe

VARIABILI LOCALI

Le variabili locali sono memorizzate nello stack

- ▶ Definite all'interno di metodi (parametri) o blocchi di codice

Ne esiste una copia diversa in ogni metodo o blocco di codice

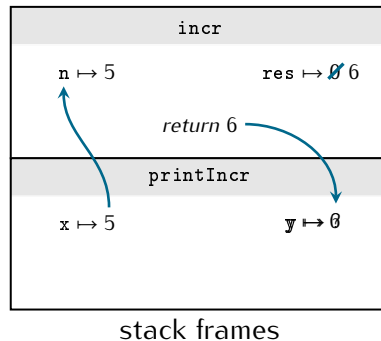
- ▶ Create all'inizio del metodo o del blocco di codice
- ▶ Distrutte automaticamente alla fine del metodo o del blocco di codice

Le variabili locali seguono la visibilità dello **scope** di blocco

// nessun oscuramento di scope annidati

STACK FRAME

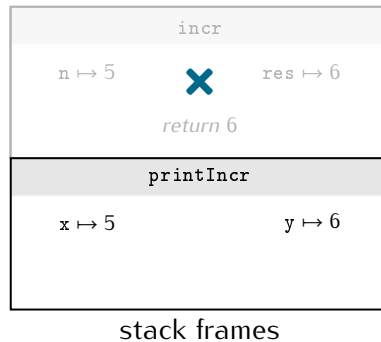
- `int incr(int n) {`
 - `int res = 0;`
 - `res = n + 1;`
 - `return res;`
 - `}`
- `void printIncr(int x) {`
 - `int y = incr(x);`
 - `System.out.println(y);`
 - `}`



Qui `n` è il **parametro formale** (variabile), mentre 5 è il **parametro attuale** (valore)

STACK FRAME (CONT.)

```
int incr(int n) {  
    int res = 0;  
    res = n + 1;  
    return res;  
}  
  
void printIncr(int x) {  
    int y = incr(x);  
    ● System.out.println(y);  
}
```



Qui `n` è il **parametro formale** (variabile), mentre 5 è il **parametro attuale** (valore)

VARIABILI STATICHE

Le variabili statiche sono memorizzate nella memoria statica

- Definite come campi `static` o costanti

La stessa copia è condivisa da tutte le istanze di una classe

- Create al caricamento della classe

Le variabili statiche possono essere accedute concorrentemente da oggetti della stessa classe

BLOCCO DI CODICE STATICO

Le variabili statiche sono inizializzate durante la dichiarazione con singoli valori

- L'inizializzazione con più istruzioni è consentita usando **blocchi di codice statici**

```
static int[] arr; // questo deve venire prima

static {
    arr = new int[3];
    arr[0] = 1;
    arr[1] = 2;
    arr[2] = 0;
}
```

1
2
3
4
5
6
7
8

I campi e i blocchi statici sono risolti dalla JVM nell'ordine della loro dichiarazione

GARBAGE COLLECTION

Il garbage collector rilascia automaticamente la memoria inutilizzata

- ▶ Controlla periodicamente la memoria cercando riferimenti inattivi
- ▶ Non ha effetto sulle variabili statiche e di istanza
- ▶ È possibile risvegliare il garbage collector con `System.gc()`

// nessuna garanzia

```
public static void main(String[]) {  
    Car myCar = new Car();  
    myCar = null;  
    System.gc(); // l'oggetto viene probabilmente deallocato  
}
```

1
2
3
4
5

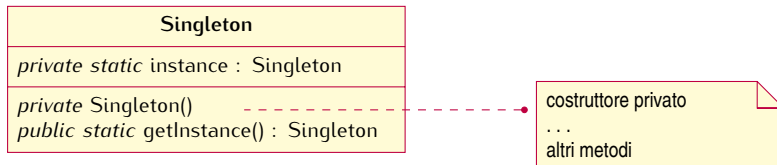
PATTERN SINGLETON

Contesto: Una classe rappresenta un concetto che richiede una singola istanza

Problema: I client potrebbero usare questa classe in modo inappropriato

Soluzione: Forzare ad avere una singola istanza della classe in memoria (singleton)

PATTERN SINGLETON (CONT.)



Limita l'istanziamento della classe a una singola istanza dell'oggetto

La classe ha un metodo che restituisce il singolo oggetto

CLASSE SINGLETON

```
public RingOfPower {                                1
    private static String owner = "Sauron";         2
    private static RingOfPower instance;            3

    private RingOfPower() { }                        4

    public static RingOfPower getInstance() {        5
        if (instance == null) { instance = new RingOfPower(); } 6
        return instance;                            7
    }                                                8
                                                    9
    public String getOwner() { return owner; }       10
}                                                    11
                                                    12
                                                    13
```

Per usare la classe: `RingOfPower.getInstance()`

OGGETTI IMMUTABILI



IMMUTABILITÀ

Immutabilità: lo stato interno degli oggetti rimane costante dopo la creazione

► L'esempio classico è la classe `String`

```
String name = "Samuel";           1
String newName = name.replace("uel", ""); 2
                                     3
System.out.println(newName); // l'output sarà 'Sam' 4
System.out.println(name);      // l'output sarà 'Samuel' 5
```

Le classi immutabili forniscono metodi di **sola lettura**

IMMUTABILITÀ (CONT.)

La parola chiave `final` consente di congelare una variabile con il suo valore di inizializzazione

► Dopo `final Person sam = new Person("Sam");` il valore di `sam` non può cambiare

Tuttavia, solo il riferimento non può cambiare, lo stato interno dell'oggetto sì

```
final Person sam = new Person("Sam");           1
                                                    2
sam = new Person("Joe"); // non consentito, il riferimento non può cambiare  3
sam.name = "Joe"; // consentito                  4
```

OGGETTI IMMUTABILI

Rendere **final** i campi di un oggetto immutabile

- ▶ Ad esempio, il campo `name` della classe `Person`

Inizializzare i campi all'interno dei costruttori della classe

```
class Person {
    private final String name; // inizializzazione nel costruttore
    Person(String name) { this.name = name; }
    public String getName() { return name; } // getter di name consentito
    // setter di name non consentito
```

1
2
3
4
5
6
7
8
9

MUTABILITÀ E IMMUTABILITÀ

Oggetti mutabili

Oggetti immutabili

Modificabilità
Thread safety
Prevedibilità
Prestazioni

Modificabili dopo la creazione
Possono richiedere sincronizzazione
Meno comprensibili
Generalmente più lenti

Rimangono costanti una volta creati
Intrinsecamente thread-safe
Più facili da debuggare
Generalmente più efficienti

ALIASING E MUTABILITÀ

Si consideri l'implementazione mutabile di Person

```
Person sam = new Person("Sam");           1
Person joe = sam; // joe e sam puntano allo stesso oggetto 2
                                                    3
joe.name = "Joe";                             4
                                                    5
                                                    6
System.out.println(sam + "␣:␣" + joe); // produce 'Joe : Joe' 7
```

Effetto collaterale: quando modifichiamo l'oggetto `joe` modifichiamo anche l'oggetto `sam`

ALIASING E MUTABILITÀ (CONT.)

Si consideri l'implementazione immutabile di Person

```
Person sam = new Person("Sam");           1
Person joe = sam; // joe e sam puntano allo stesso oggetto 2
                                                    3
joe.name = "Joe"; // errore di compilazione 4
joe = new Person(                             5
                                                    6
System.out.println(sam + "␣:␣" + joe); // produce 'Sam : Joe' 7
```

Nessun effetto collaterale

PASSAGGIO DI ARGOMENTI AI METODI

Come vengono passati i parametri dal chiamante al chiamato?

Passaggio per valore

```
class Clazz { // il parametro attuale viene copiato nel parametro formale 1
    void aMethod() { // chiamante 2
        int x = 5; 3
        int y = increment(x); 4
        System.out.println(x + " : " + y); 5
    } 6
    int increment(int n) { // chiamato 7
        return ++n; 8
    } 9
    // x ed n sono variabili diverse 10
}
```

parametro attuale

parametro formale

qui viene stampato '5 : 6'

11
12
13
14

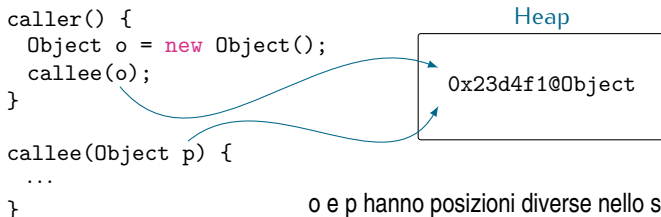
PASSAGGIO DI ARGOMENTI IN JAVA

Java usa **sempre** il passaggio per valore

- Va bene per variabili primitive, ma è insidioso per variabili oggetto

Il valore di una variabile oggetto è un riferimento, il valore copiato è un riferimento

- Parametro attuale e formale puntano allo stesso oggetto in memoria



PASSAGGIO DI ARGOMENTI INSIDIOSO

```
class Clazz { // passaggio per valore      1
    void aMethod() { // chiamante          2
        Person samuel = new Person("Samuel"); 3
        Person sam = shortenName(samuel);      4
        System.out.println(samuel + "␣:␣" + sam); viene stampato 'Sam : Sam' 5
    }                                           6
    Person shortenName(Person p) { // chiamato 7
        p.name = p.name.substring(0, 3);      8
        return new Person(p.name);           9
    }                                         10
}                                           11
                                           12
                                           13
                                           14
                                           15
                                           16
```


PASSAGGIO DI ARGOMENTI INSIDIOSO (CONT.)

```
class Clazz { // passaggio per valore      1
    void aMethod() { // chiamante          2
        Person samuel = new Person("Samuel"); 3
        Person sam = shortenName(samuel);      4
        System.out.println(samuel + "␣:␣" + sam); viene stampato 'Samuel : Sam' 5
    }                                           6
    Person shortenName(Person p) { // chiamato 7
        p = new Person(p.name);              8
        p.name = p.name.substring(0, 3);     9
        return p;                            10
    }                                         11
}                                           12
                                           13
                                           14
                                           15
                                           16
```

CLASSI WRAPPER



PURO ORIENTAMENTO AGLI OGGETTI

In un linguaggio a oggetti puro, esistono solo classi e oggetti

- ▶ Per motivi di efficienza, Java usa tipi primitivi (`int`, `float`, ...)

Le classi wrapper sono versioni a oggetti dei tipi primitivi

- ▶ Definiscono operazioni di conversione tra tipi diversi

CLASSI WRAPPER

Tipo primitivo	Classe wrapper
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Char
boolean	Boolean

I costruttori delle classi wrapper creano oggetti che contengono un valore primitivo

- Le classi wrapper sono definite nel package `java.lang`

CLASSI WRAPPER (CONT.)

Per effettuare il wrapping di un valore `int`

▶ `Integer wi = new Integer(3);`

// boxing

▶ Per estrarlo: `int i = wi.intValue();`

// unboxing

Per effettuare il wrapping di un valore `double`

▶ `Double wd = new Double(2.1);`

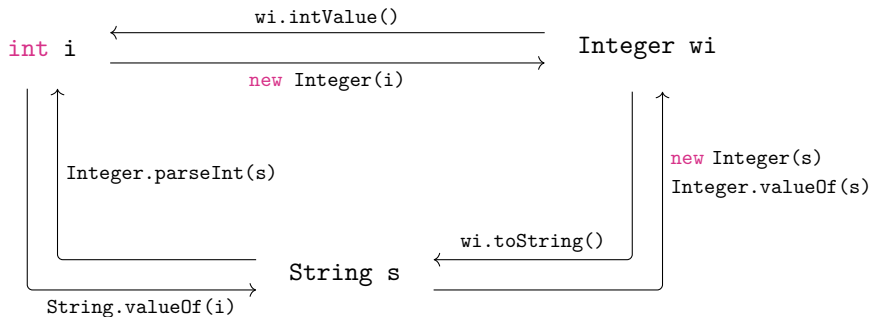
// boxing

▶ Per estrarlo: `double d = wd.doubleValue();`

// unboxing

Qui `wi` e `wd` sono riferimenti a oggetti contenenti valori primitivi

CONVERSIONI



AUTOBOXING

A partire da Java 5, il compilatore esegue boxing e unboxing automatici

```
Integer wi = new Integer(2);  
int j = 0;  
  
j = wi + 5; // invece di j = wi.intValue() + 5;  
  
wi = 3;      // invece di wi = new Integer(3);  
  
wi = j;      // invece di wi = new Integer(j);
```

1
2
3
4
5
6
7
8

In effetti, `new Integer()` è deprecato a partire da Java 9

METODI DI UTILITÀ

Le classi wrapper forniscono metodi di utilità

La classe `Character` fornisce alcuni utili metodi `static`

- ▶ `isLetter()`, `isDigit()`, `isSpaceChar()`, . . .
- ▶ `toUpperCase()`, `toLowerCase()`, . . .

```
Character wc1 = 'g';           // usare "g" causerà un errore di compilazione    1
Character wc2 = Character     2
                                3
System.out.println(wc2); // l'output sarà 'G'                                     4
System.out.println(wc1); // l'output sarà 'g'                                     5
```


COME UNA STRINGA

Le classi wrapper sono semplici contenitori di valori, come la classe `String`

- ▶ Gli oggetti delle classi wrapper sono immutabili, come gli oggetti `String`
- ▶ Le classi wrapper dispongono di un metodo `toString()`

```
int j = 3; 1
Integer wi = j; // viene creato un nuovo oggetto Integer '3', wi punta ad esso 2
wi = 7;      // viene creato un nuovo oggetto Integer '7', wi punta ad esso 3
```

INSIDIE

Il confronto tra classi wrapper con `==` verifica l'uguaglianza dei riferimenti

- Usare il metodo predefinito `equals()`

```
int x = 3;                                     1
Integer wx = x;                                2
int y = 3;                                     3
Integer wy = y;                                4
Integer wi = new Integer(3);                   5

System.out.println(x == y);                    6
System.out.println(wx == wi);                  7
System.out.println(wx == wy);                  8
System.out.println(wx.equals(wy));             9
System.out.println(wx.equals(wi));            10
System.out.println(wx.equals(wi));            11
```

// l'output sarà 'true'
// l'output sarà 'false'
// l'output potrebbe essere 'true'
// l'output sarà 'true'
// l'output sarà 'true'

INSIDIE (CONT.)

Il metodo `Integer.parseInt()` accetta una stringa e restituisce il valore numerico

- La stringa deve rappresentare un valore intero valido

```
int i = Integer.parseInt("13") + 3; // l'output sarà '16'      1
int j = Integer.parseInt("␣13");    // la JVM lancerà un'eccezione 2
int k = Integer.parseInt("2.1");     // la JVM lancerà un'eccezione 3
```



DOMANDE E RISPOSTE

